

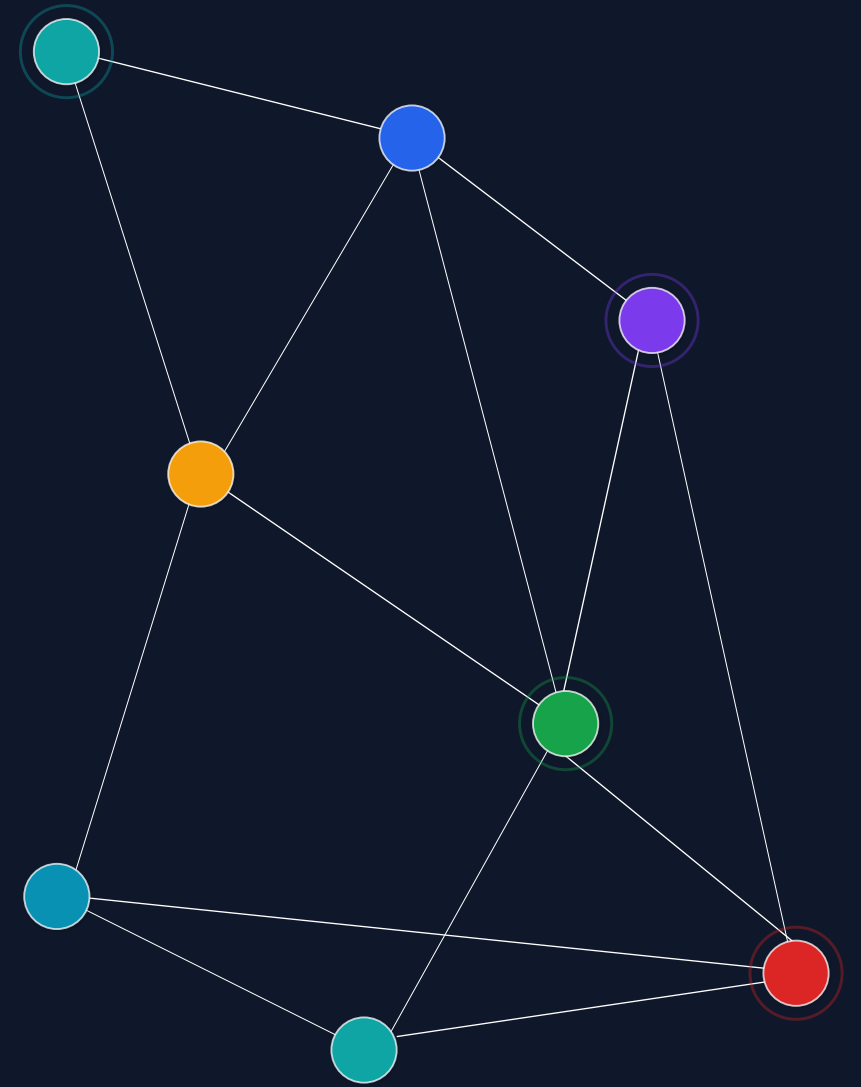
Middleware pour le Big Data

Séance 1 — Comprendre, choisir et architecturer une plateforme data moderne

2026

Tariq LAMBACHRI

• Big Data • Middleware



Ce que vous saurez faire à la fin de la séance

Des objectifs observables, évaluables et directement liés aux ateliers.

1 Expliquer

Décrire les 5V, le rôle d'un middleware et les contraintes d'une plateforme Big Data.

2 Comparer

Choisir entre Kafka, Airflow, Spark, Flink, Talend, NoSQL ou Lakehouse selon le besoin.

3 Concevoir

Dessiner une architecture cible avec flux, stockage, traitement, sécurité et observabilité.

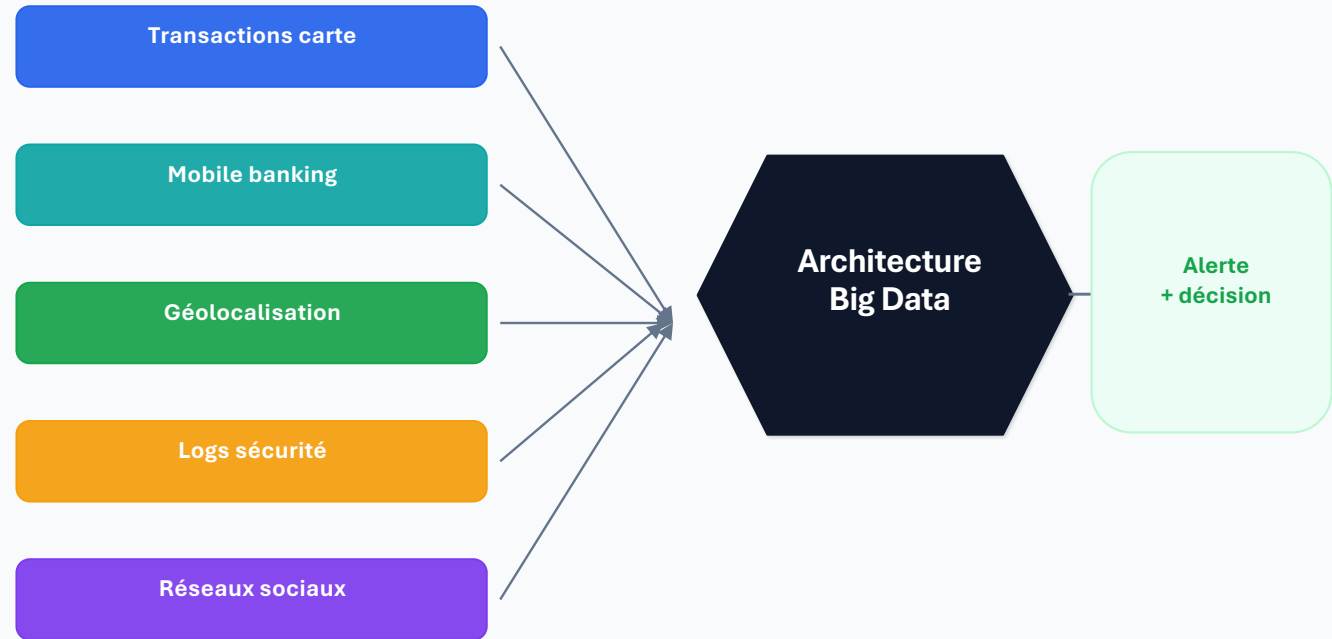
Livrable : une architecture argumentée en 1 page + une restitution orale de 3 minutes.

Problème déclencheur : détecter une fraude en temps réel

Un cas métier simple pour relier les concepts techniques à une décision d'architecture.

Le besoin métier

Une banque veut repérer des transactions suspectes en moins de 100 ms, sans bloquer les paiements légitimes, et avec traçabilité complète des décisions.



Question centrale : où placer l'ingestion, le traitement, le stockage, la gouvernance et les garanties de fiabilité ?

Pourquoi le Big Data aujourd'hui ?

Le sujet n'est pas seulement la quantité : c'est la combinaison volume + temps + confiance.

175 ZB

ordre de grandeur de la Global Datasphere
prévue pour 2025

Sources hétérogènes

applications, IoT, logs, capteurs, documents, médias

Décisions plus rapides

alertes, recommandations, détection d'anomalies, monitoring

Contraintes plus fortes

sécurité, RGPD, qualité, audit, coûts cloud

Message clé : le Big Data devient un problème d'architecture distribuée, pas seulement un problème de stockage.

Les 5V : un diagnostic, pas une définition à apprendre par cœur

Chaque V donne un indice sur les choix d'architecture.

Volume

Combien ?
stockage distribué, coût, partitionnement

Vélocité

À quelle vitesse ?
streaming, latence, backpressure

BIG
DATA

Variété

Quels formats ?
schéma, connecteurs, formats

Véracité

Quelle confiance ?
qualité, validation, lineage

Valeur

Quelle décision ?
KPI, ML, produit data, ROI

Atelier 1 — Diagnostiquer un cas avec les 5V

8 minutes de travail + 4 minutes de restitution rapide.

1 Choisissez un domaine

santé, transport, banque, e-commerce, industrie, smart city

2 Identifiez les données

sources, formats, fréquence, volumes, acteurs

3 Reliez chaque V à un risque

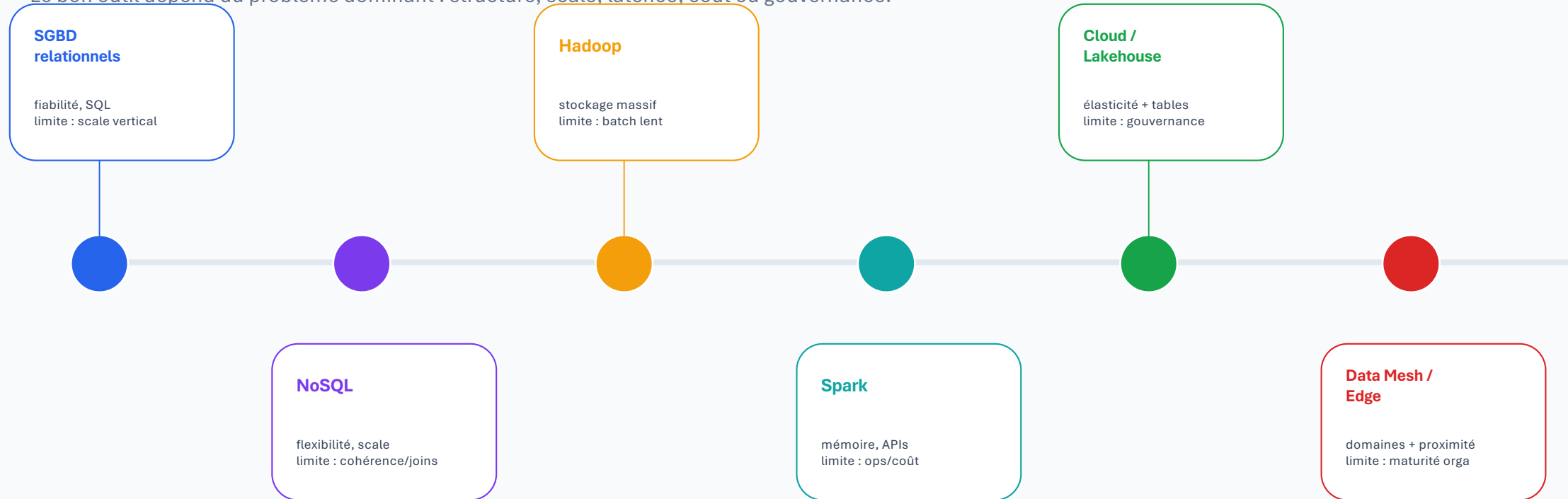
latence, coût, qualité, sécurité, adoption métier

4 Proposez un premier middleware

Kafka ? Airflow ? Talend ? Spark ? autre ?

Évolution des systèmes de données : chaque étape répond à une limite

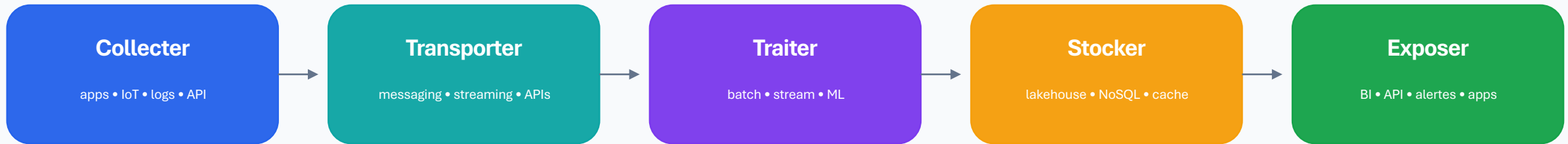
Le bon outil dépend du problème dominant : structure, scale, latence, coût ou gouvernance.



À retenir : le Big Data moderne combine souvent plusieurs générations dans la même architecture.

De la donnée brute à la décision : où intervient le middleware ?

Une plateforme data est une chaîne de responsabilités, pas une pile de logos.



Couches transverses : sécurité • gouvernance • observabilité • qualité • coûts

Middleware = connexion + coordination + fiabilité

C'est quoi un middleware ?

Une couche logicielle qui rend les systèmes capables de travailler ensemble.

Un middleware connecte, transforme, orchestre et sécurise les échanges entre applications, services et données.

Connecter

protocoles, API, connecteurs, formats

Découpler

buffer, file, topic, reprise sur incident

Orchestrer

DAG, dépendances, scheduling,
retries

Gouverner

sécurité, qualité, monitoring, lineage

Exemples : Kafka, RabbitMQ, Airflow, NiFi, Spark, Flink, Talend, API Gateway, gRPC.

Typologie des middlewares Big Data

Classer par fonction avant de classer par technologie.

Famille	Exemples	Question à poser
Messaging / Event streaming	Kafka, RabbitMQ, Pulsar	Transport asynchrone, découplage, replay
Orchestration / Workflow	Airflow, NiFi	Planifier, dépendances, reprises, SLA
Traitement distribué	Spark, Flink	Calcul batch, micro-batch, streaming stateful
Intégration / ETL / ELT	Talend, Informatica, dbt	Connecter, transformer, industrialiser
Serving / API / Cache	API Gateway, Redis, gRPC	Exposer données et services à faible latence

Erreur fréquente : “on met Kafka partout”. Bon réflexe : partir du besoin de flux, de latence, de fiabilité et de gouvernance.

Architecture de référence d'une plateforme Big Data moderne

Un modèle en couches pour lire n'importe quelle architecture réelle.



Zoom technologique

Apache Kafka et le raisonnement par flux



Pourquoi Kafka ?

Kafka est utile quand il faut absorber, découpler et rejouer des événements.

Absorber les pics

Une file durable évite que chaque service doive encaisser le pic en direct.

Découpler les services

Les producteurs n'ont pas besoin de connaître tous les consommateurs.

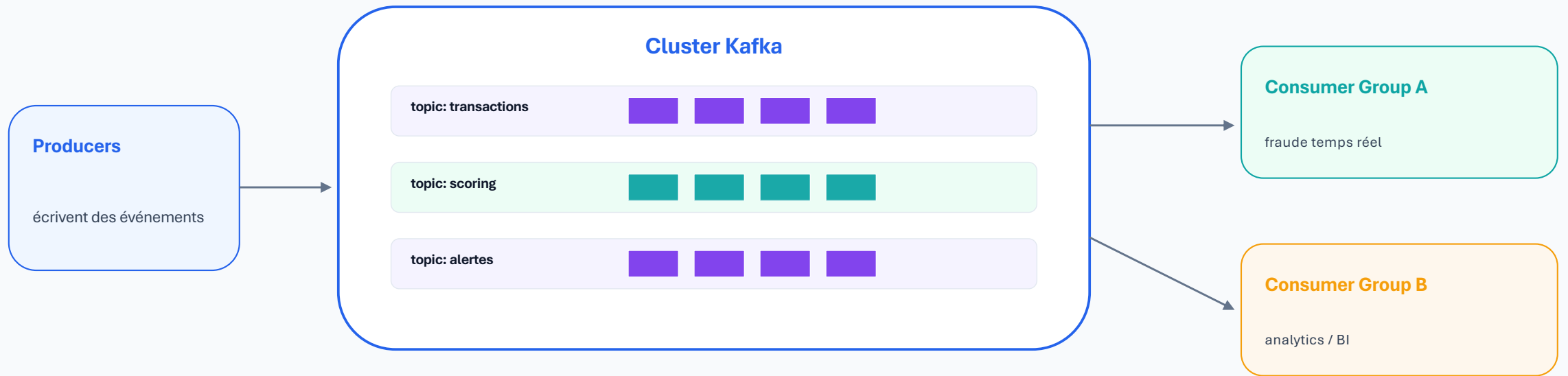
Rejouer l'historique

La rétention permet de reconstruire un état ou d'alimenter un nouveau service.

À retenir : Kafka transporte des événements ; il ne remplace ni une base de données, ni un orchestrateur, ni un moteur de traitement complet.

Kafka : modèle mental minimal

Cinq concepts suffisent pour concevoir un premier flux.



Topic = catégorie d'événements

Partition = unité de parallélisme

Offset = position de lecture

Broker = nœud du cluster

Consumer group = lecture parallèle + isolation

Kafka moderne

Ancienne représentation

Producteurs → Brokers Kafka
+ ZooKeeper pour la coordination

Utile historiquement, mais cela donne une vision datée pour les nouveaux déploiements.



Représentation actuelle

Producteurs → Cluster Kafka en mode KRaft
Contrôleurs Kafka intégrés

Moins de composants externes, architecture de contrôle simplifiée.

Concevoir un flux Kafka : 5 décisions à justifier

Chaque décision a un impact sur la performance, la fiabilité et l'exploitation.

1 Découper les topics

événements métier, granularité, conventions de nommage

3 Dimensionner les partitions

parallélisme, débit, rééquilibrages

5 Valider les schémas

compatibilité, évolution, contrats de données

2 Choisir les clés

ordre par entité, partitionnement, hotspots

4 Définir la rétention

rejeu, coûts, obligations légales

Fiabilité : que signifie “ne pas perdre la donnée” ?

La fiabilité est une combinaison de garanties, de configuration et de monitoring.

At-most-once

On peut perdre un événement, mais on ne le traite pas deux fois.
Cas : métriques non critiques.

At-least-once

On ne perd pas l'événement, mais des doublons sont possibles.
Cas : pipelines analytiques.

Exactly-once*

Effet unique observé dans une chaîne contrôlée.
Cas : traitements transactionnels.

Design checklist : acknowledgements • idempotence • transactions • déduplication • replay • monitoring du lag.

Atelier 2 — Concevoir un modèle de topics Kafka

15 minutes en binômes : passer du cas métier au modèle d'événements.

Sources

transactions, mobile, logs, géolocalisation

Topics

nommage, granularité, événement métier

Clés

client_id, card_id, merchant_id, device_id

Consumers

scoring fraude, alerting, data lake, audit

Risques

latence, doublons, schéma, données sensibles

Restitution attendue : un mini schéma + 3 décisions justifiées.

Traitement & orchestration

Batch, streaming, DAG et intégration



Batch, micro-batch, streaming : choisir selon la latence métier

La question n'est pas "quelle techno est meilleure ?" mais "quel délai est acceptable ?".

Batch

minutes → heures
rapports, backfills, agrégats historiques

Micro-batch

secondes → minutes
streaming structuré, agrégats quasi temps réel

Streaming stateful

millisecondes → secondes
fraude, IoT critique, alertes opérationnelles

moins cher / plus simple

latence plus faible / ops plus exigeantes

Spark ou Flink ? Deux moteurs, deux points forts

Un comparatif volontairement simplifié pour débiter l'analyse d'architecture.

Apache Spark

- Traitement distribué très utilisé
- Fort en batch, SQL, ML, lakehouse
- Structured Streaming : calcul incrémental sur Spark SQL
- Bon choix si l'entreprise a déjà des jobs Spark et un data lake

Apache Flink

- Moteur conçu pour flux bornés et non bornés
- Fort en streaming stateful, event time, faible latence
- Checkpoints pour cohérence et reprise
- Bon choix si le temps réel opérationnel est critique

Règle simple : Spark pour l'unification batch/analytique ; Flink pour le streaming stateful exigeant.

Airflow : orchestrer des workflows, pas transporter chaque événement

Airflow complète Kafka, Spark et Flink ; il ne joue pas le même rôle.

Airflow

Planifie et exécute des workflows finis : DAGs, tasks, dépendances, retries, SLA, monitoring.

Kafka / Flink

Gère des flux continus ou des calculs streaming : événements, état, latence faible, backpressure.

Ensemble

Kafka ingère → Flink/Spark traite → Airflow déclenche backfill, validation, reporting ou tâches batch.

Question d'examen possible : “Pourquoi Airflow n’est-il pas un bon choix pour traiter chaque message d’un topic Kafka ?”

Stockage : du HDFS au Lakehouse

La tendance moderne : conserver l'économie du data lake tout en récupérant des garanties de table.

Fichiers

Parquet • ORC • Avro

format physique efficace

Stockage objet

S3 • ADLS • GCS • HDFS

scalabilité et coût

Table format

Delta Lake • Iceberg • Hudi

ACID, schéma, time travel

Moteurs

Spark • Flink • Trino • BI

requêtes et traitements

Idée clé : le “middleware data” doit aussi parler aux formats de table, pas seulement aux bases.

NoSQL et bases spécialisées : choisir selon le modèle d'accès

Le critère principal n'est pas le buzzword, mais la requête dominante.

Document

MongoDB
JSON, contenu variable, catalogue, profil utilisateur

Key-value / cache

Redis
latence faible, session, cache, feature flags

Wide-column

Cassandra / HBase
écriture massive, time-series large échelle

Graph

Neo4j
relations, fraude, recommandation, réseaux

Time-series

InfluxDB / Prometheus
métriques, IoT, monitoring

Question de design : “quelle requête doit être rapide et sûre ?”

ETL, ELT et intégration : industrialiser les flux

Le middleware d'intégration rend les pipelines maintenables, observables et réutilisables.

ETL

Extract → Transform → Load

Transformation avant chargement. Utile quand on contrôle fortement le format cible.

ELT

Extract → Load → Transform

Transformation dans le data warehouse/lakehouse. Utile avec le cloud et SQL analytique.

CDC

Change Data Capture

Capturer les changements d'une base pour alimenter des flux ou une plateforme analytique.

Talend / NiFi / Informatica / dbt : mêmes familles de problèmes, niveaux d'abstraction différents.

Gouvernance & qualité : les données doivent être utilisables et défendables

Un pipeline qui tourne n'est pas forcément un pipeline fiable.

Catalogue

où est la donnée ? qui la possède ?

Lineage

d'où vient-elle ? quelles transformations ?

Qualité

complétude, fraîcheur, validité, unicité

Conformité

PII, consentement, minimisation, audit

Contrats

schéma, SLA, conventions, responsabilités

Dans un projet réel, ces points doivent être décidés au début, pas ajoutés à la fin.

Sécurité by design : chaque couche a ses contrôles

Le middleware multiplie les échanges ; il faut donc maîtriser les accès et les traces.

Identité

authentification, SSO, service accounts

Autorisation

RBAC/ABAC, ACL topics, least privilege

Chiffrement

TLS en transit, chiffrement au repos

Secrets

rotation, vault, pas de clés dans les jobs

Audit

logs, lineage, alerting, détection d'anomalies

Réseau

segmentation, private links, zero trust

Exercice rapide : où placer un contrôle d'accès dans l'architecture de référence ?

Observabilité data : voir les pannes techniques et les pannes de sens

Un système peut être “UP” tout en livrant des données fausses ou trop tardives.

Technique

latence • débit • erreurs • CPU/RAM • saturation

Flux

Kafka lag • retries • backpressure • files en attente

Données

fraîcheur • volume attendu • nulls • valeurs hors plage

Métier

KPI affectés • alertes inutiles • décisions retardées

SLO possible : 99 % des transactions scorées en moins de 100 ms, avec données fraîches < 5 s.

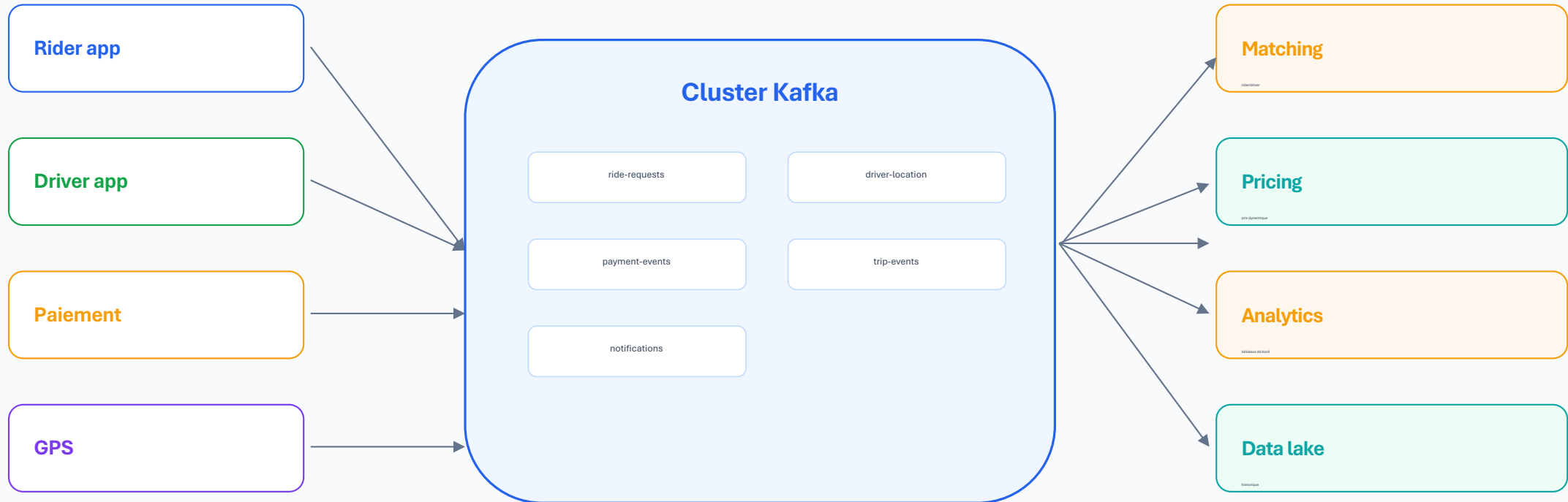
Cas pratiques

Lire et construire une architecture



Cas d'usage : mobilité à la demande

Une version neutre du cas Uber pour comprendre la logique des flux.



Utilisation pédagogique : demander quelles données doivent être temps réel, lesquelles peuvent être batch, et où placer la gouvernance.

Cas fil rouge : IoT + Banque + Réseaux sociaux

Le sujet de l'atelier final reprend les trois domaines de ton support initial.

Sources

- Capteurs IoT : mesures et alertes
- Banque : transactions, paiements, authentification
- Réseaux sociaux : posts, likes, sentiment
- Web/mobile : événements utilisateur

Contraintes

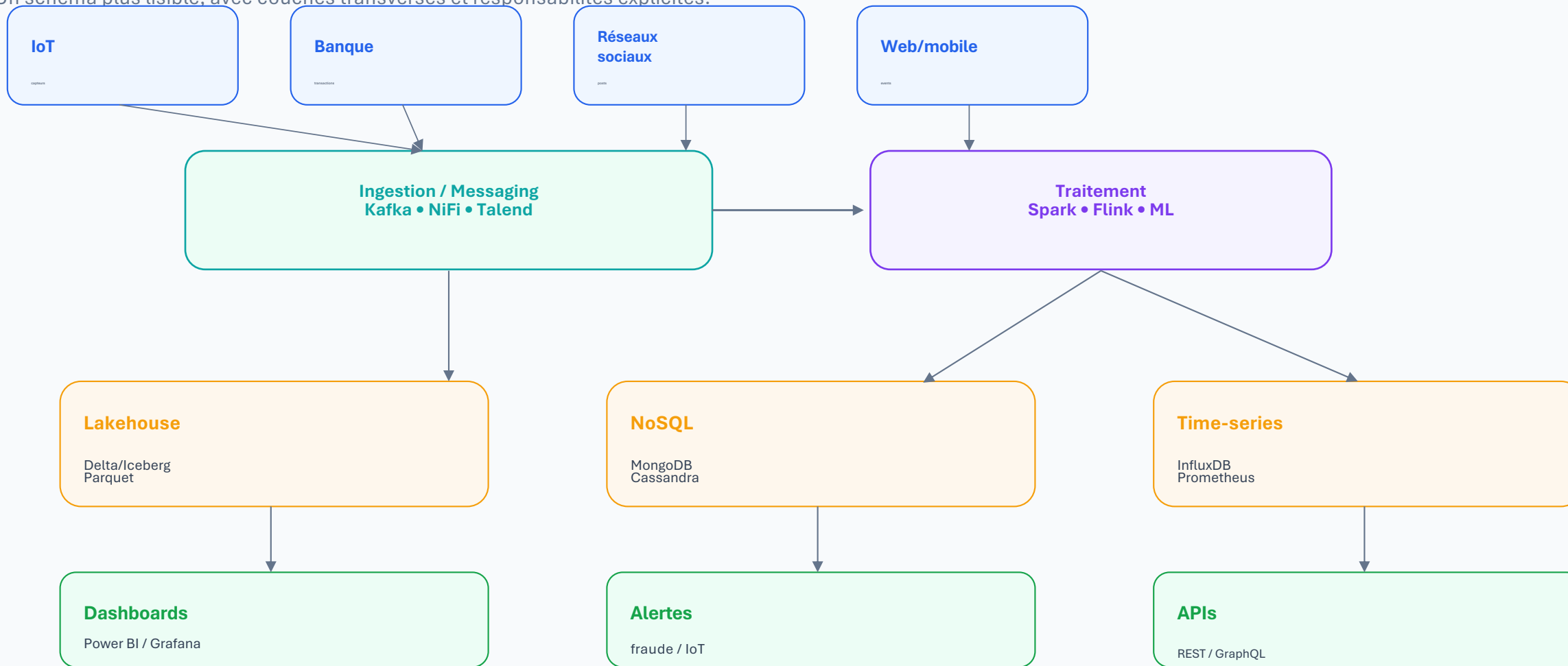
- Certaines données en temps réel
- Données sensibles et auditable
- Volumes variables et pics
- Besoin de BI + alertes + ML
- Coûts cloud à maîtriser

Mission

Construire une architecture en couches et justifier chaque middleware choisi.

Architecture cible — proposition corrigée et enrichie

Un schéma plus lisible, avec couches transverses et responsabilités explicites.



Atelier 3 — Construire l'architecture en groupes

30 minutes de conception + 3 minutes de restitution par groupe.

1. Schéma

couches, composants, flux, données sensibles

2. Justification

pourquoi Kafka/Spark/Flink/Talend/NoSQL/etc. ?

3. Exploitation

monitoring, erreurs, reprise, coût, sécurité

4. Compromis

ce que l'architecture ne résout pas encore

Règle : aucun logo sans rôle explicite dans l'architecture.

Grille d'évaluation rapide de l'atelier

Une grille simple pour rendre les attentes explicites.

Critère	Attendus	Points
Architecture en couches	flux lisibles, responsabilités, cohérence globale	4 pts
Choix middleware	Kafka/Airflow/Spark/Flink/Talend justifiés par contraintes	5 pts
Stockage & traitement	modèles d'accès, batch/streaming, types de bases	4 pts
Sécurité & gouvernance	PII, accès, qualité, audit, lineage	4 pts
Clarté de restitution	schéma clair, compromis assumés, vocabulaire précis	3 pts

Total : /20 — utilisable comme note formative ou bonus de participation.

Talend en un coup d'œil : où le placer dans l'architecture ?

Un outil d'intégration qui peut générer et exécuter des pipelines sur des environnements Big Data.

Connecteurs

JDBC, fichiers, cloud, Hadoop/Spark, APIs

Studio visuel

jobs graphiques, composants, transformations

Exécution

modes Spark selon cluster et configuration

Qualité

contrôles, profils, validation, règles

Industrialisation

logs, monitoring, déploiement, réutilisation

Positionnement : Talend est surtout utile pour intégrer et industrialiser, pas pour remplacer toute la plateforme.

Quiz de sortie — 10 questions rapides

1. Quel V influence le plus la latence ?

3. Pourquoi utiliser un consumer group ?

5. Airflow est-il adapté à un flux continu message par message ?

7. À quoi sert un table format type Delta/Iceberg ?

9. Où placer la sécurité dans l'architecture ?

2. Différence entre topic et partition ?

4. Kafka remplace-t-il une base de données ?

6. Quand préférer Flink à Spark ?

8. Donnez deux métriques d'observabilité data.

10. Citez un compromis de votre architecture.

Option : réutiliser le QR code Quizizz du support initial après mise à jour des questions.

Tendances à surveiller

Pour montrer que le middleware Big Data évolue aussi côté organisation et production.

Data Mesh

responsabilité par domaine, data products, gouvernance fédérée

Lakehouse & table formats

Delta, Iceberg, Hudi : fiabilité au-dessus du data lake

Edge + IoT

traiter près de la source quand la latence ou le réseau l'impose

Data contracts

schémas, SLA, qualité et responsabilités explicites

Observabilité data

freshness, volume, schema drift, lineage, alerting

IA générative & données

RAG, vector stores, gouvernance de connaissances

Message : la technologie seule ne suffit pas ; les modèles d'organisation et de gouvernance sont devenus centraux.

Conclusion

4 messages à retenir

1

Le Big Data est un problème de contraintes : volume, vitesse, variété, confiance et valeur.

2

Le middleware connecte, découple, orchestre, sécurise et rend les flux exploitables.

3

Kafka, Spark/Flink, Airflow, Talend et les stockages ne se remplacent pas : ils se complètent.

4

Une architecture professionnelle intègre gouvernance, sécurité, qualité et observabilité dès le départ.

Séance 2 : intégration de données et premiers pipelines